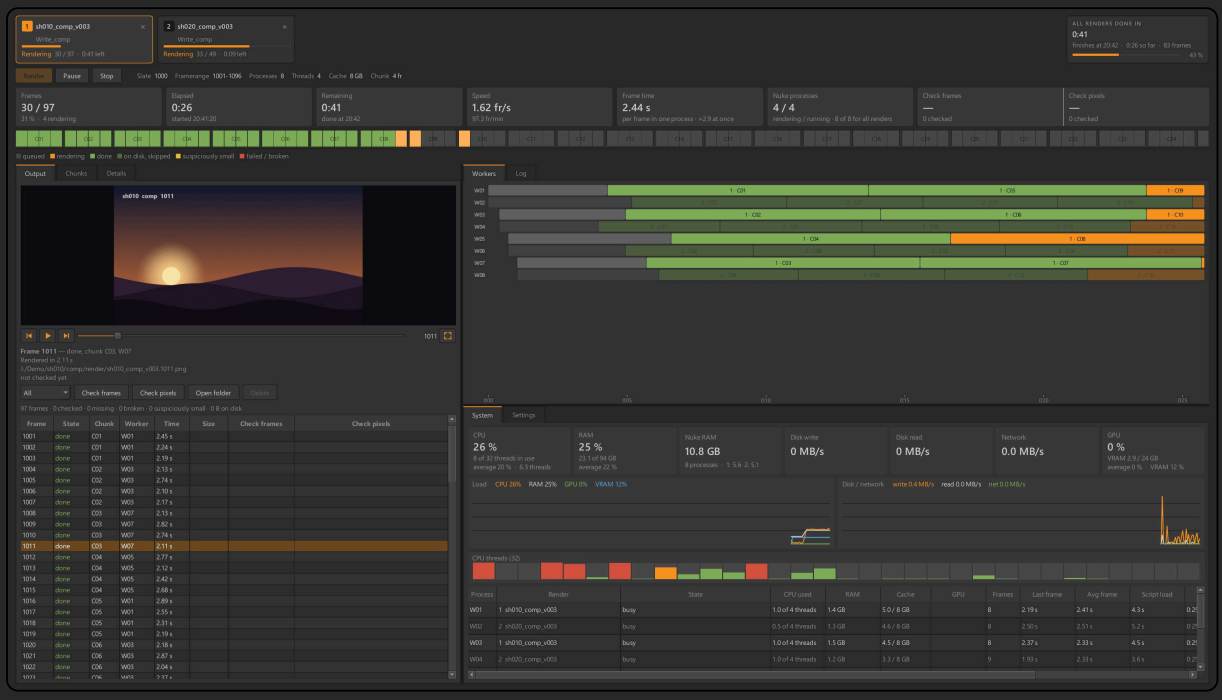
INSTRUKCJA · WERSJA 0.94.3 ALPHA

JKrender

Szybki lokalny render dla Nuke. Render dzieli się na kawałki (chunki), które liczy jednocześnie kilka procesów Nuke. Po renderze JKrender sprawdza każdą klatkę na dysku.

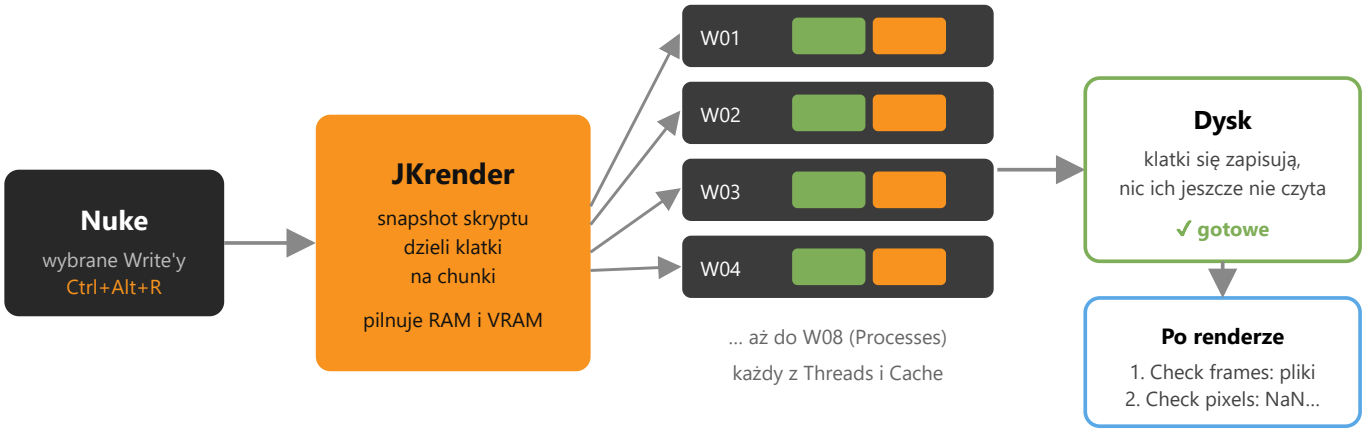


Spis treści

1 Jak działa JKrender
2 Instalacja i uruchomienie z Nuke
3 Mapa okna
4 Karty renderów i wiersz sterowania
5 Kafelki i mapa klatek
6 Panel Output: podgląd i klatki
7 Panele Chunks i Details
8 Panele Workers i Log
9 Panel System
10 Settings: każde ustawienie
11 Jak ustawić render
12 Test this script
13 Pilnowanie RAM i VRAM
14 Kontrole, dźwięki, awarie, dane
15 Mysz i klawisze: ściąga
16 Gdy coś nie działa

1 Jak działa JKrender

Zwykły render w Nuke działa w jednym procesie, który klatka po klatce liczy cały skrypt. Jeden Nuke rzadko jednak wykorzystuje wszystkie 32 wątki procesora. Dlatego JKrender uruchamia **kilka procesów Nuke obok siebie**. Każdy **otwiera skrypt tylko raz**, a potem dostaje kawałki pracy, tak zwane **chunki**: na przykład 10 klatek po kolei.



Pod koniec chunki się zmniejszają, więc wszystkie procesy kończą mniej więcej razem.

Trzy liczby, które decydują o szybkości

Processes

Ile procesów Nuke działa naraz (dla wszystkich renderów razem). Więcej procesów robi więcej naraz, ale bierze też więcej RAM i VRAM.

Threads

Ile wątków procesora dostaje każdy proces (Nuke `-m`). **Processes × Threads** powinno dać mniej więcej liczbę wątków PC (tu 32).

Cache

Limit cache jednego procesu w GB (Nuke `-c`). Proces w rzeczywistości bierze więcej. Ile, pokazuje System › RAM.

> Domyślnie jest **8 procesów × 4 wątki**. Ciężkie skrypty compu, które jeden Nuke wykorzystuje sam (Defocus, Denoise), bywają szybsze na **4 × 8** i biorą połowę RAM. Skrypty, którym rośnie VRAM, chcą mniej procesów. Co jest najlepsze dla twojego skryptu, zmierzy **Test this script** (rozdział 12).

Co gdzie jest zapisane

Co	Gdzie
Ustawienia	`~/.nuke/JKrender/settings/settings.json` (każdy użytkownik ma swoje). Starsze wersje miały je w `~/.jkrender`; JKrender przenosi je sam przy pierwszym uruchomieniu.
Logi renderów	`~/.nuke/JKrender/log/`: jeden folder na render (snapshot skryptu, job, log każdego procesu Nuke i `report.json`). Starsze niż 15 dni są usuwane.
Wyniki testów	`~/.nuke/JKrender/settings/test_results`
Wyrenderowane klatki	Tam, gdzie wysyła je Write. JKrender nigdy ich sam nie usuwa, tylko przyciskiem Delete.

2 Instalacja i uruchomienie z Nuke

Instalacja (raz)

1. Skopiuj cały folder `JKrender` do `%USERPROFILE%\.nuke\`. Pobrany z GitHuba jako zip nazywa się `JKrender-main`: zmień nazwę na `JKrender`.

2. Dodaj do `~/.nuke/init.py` tę linię (w razie potrzeby utwórz plik):

```
nuke.pluginAddPath("./JKrender")
```

3. Uruchom Nuke ponownie. Przy pierwszym starcie JKrender pyta, czy pobrać numpy i OpenEXR do podglądu EXR (około 15 MB). Jeśli odmówisz, render działa dalej, tylko bez podglądu EXR. Aby pytanie wróciło, usuń plik `lib/DECLINED`.

Opcjonalnie: dla szczegółów i odtwarzacza MOV/MP4 dodaj ffprobe/ffmpeg do PATH (`winget install ffmpeg`). Potrzebny jest Nuke 16 (PySide6).

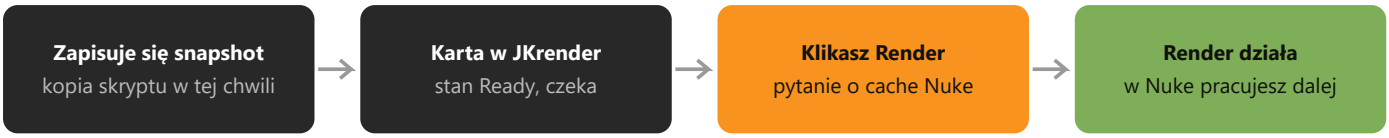
Nowa wersja: skopiuj nowe pliki na stare. Foldery `lib`, `log` i `settings` zostają.

Menu JKrender w Nuke

Pozycja	Co robi
Render Selected Write(s) `Ctrl+Alt+R`	Wysyła wybrane Write'y (lub Groupy z Write'ami) do JKrender. Każdy Write dostaje własną kartę. Jeśli okno jeszcze nie działa, otwiera się.
Open Renders Folder	Otwiera folder z logami renderów.

Gdy skrypt nie jest zapisany, pyta *Save it and render?*: renderuje się zapisana wersja.

Co się dzieje po wysłaniu

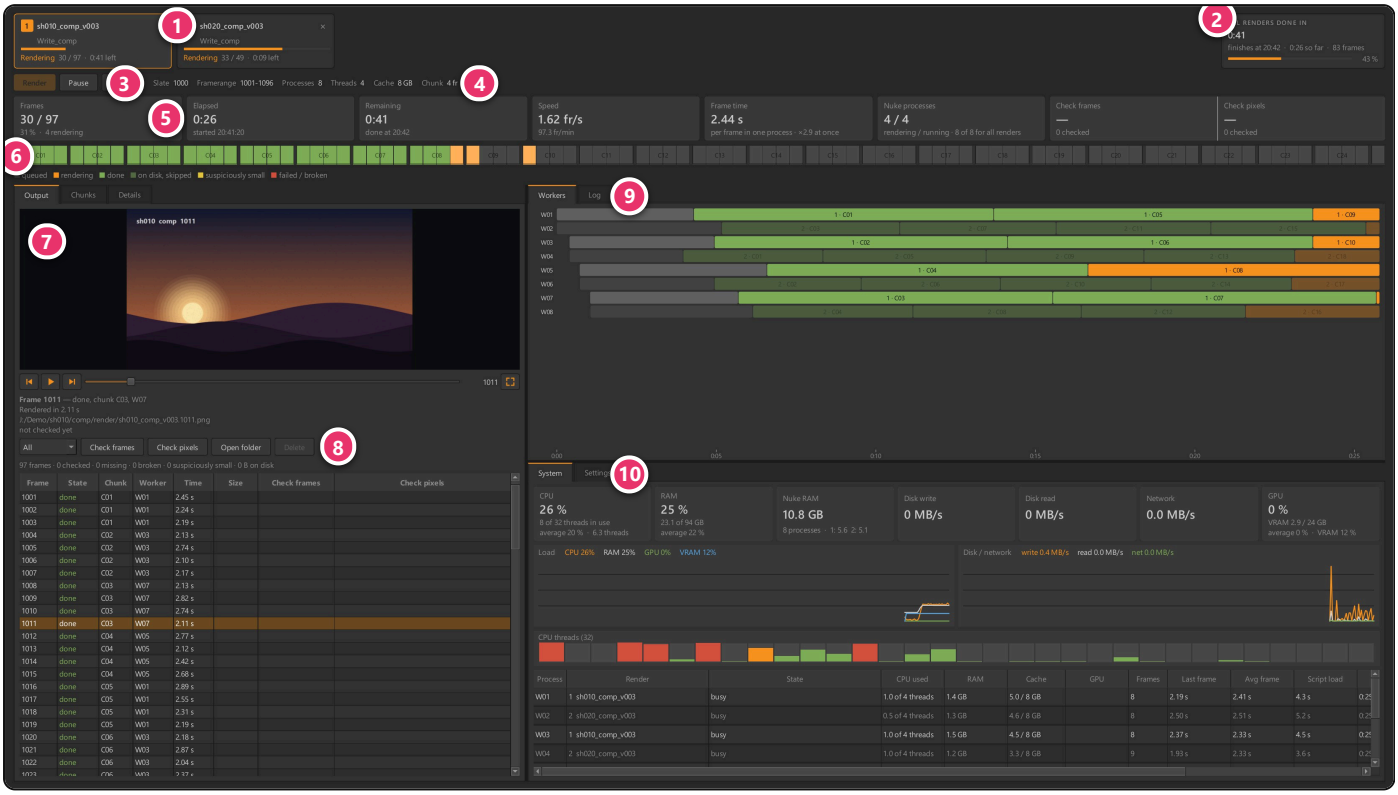


> **Snapshot:** renderuje się skrypt w takiej postaci, jaką miał przy wysłaniu. W Nuke możesz od razu dalej pracować i zapisywać, na trwający render nie ma to wpływu. Gdy zmienisz skrypt i chcesz go wyrenderować ponownie, wyślij go z Nuke jeszcze raz: nowsze wysłanie zastąpi starą kartę.

> **Pytanie o cache:** gdy jest otwarty jakiś Nuke, Render pyta, czy wyczyścić jego bufory, cache odtwarzania i cache Blink, żeby render miał więcej pamięci. Domyślna odpowiedź to **No**, a render startuje w obu przypadkach. Cache dyskowego to nie rusza. Po aktualizacji JKrender trzeba raz zrestartować Nuke, żeby to działało.

3 Mapa okna

Na górze są karty renderów i wiersz sterowania, pod nimi liczby i mapa klatek. Resztę tworzą trzy panele: po lewej **Output / Chunks / Details**, po prawej u góry **Workers / Log** i po prawej na dole **System / Settings**. Granice między panelami można przeciągać myszą.



1. Karty renderów: jeden Write (render) na kartę. Kliknięcie go wybiera.
2. Kiedy będą gotowe wszystkie rendery.
3. Render, Pause, Stop dla wybranego renderu.
4. Slate, Framerange, Processes, Threads, Cache i Chunk tego renderu (tylko do odczytu).
5. Kafelki z liczbami: klatki, czas, szybkość, procesy, kontrole.
6. Mapa klatek według chunków. Kliknięcie klatki ją pokazuje.
7. Output: podgląd klatki z odtwarzaniem.
8. Output: filtr, kontrole, folder, usuwanie i lista klatek.
9. Workers: oś czasu każdego procesu Nuke.
10. System: obciążenie PC i tabela procesów.

4 Karty renderów i wiersz sterowania

Karta renderu

Każdy wysłany Write to jedna karta: numer, nazwa skryptu, nazwa Write'a, pasek postępu i stan (Ready, Waiting, Rendering, Paused, Checking output, Done, Failed, Stopped). **Kliknięcie** wybiera kartę i całe okno pokazuje wtedy ten render. **Shift+klik** wybiera kilka kart; Render, Pause i Stop działają wtedy na wszystkie, a Render again także na gotowe (pyta raz za wszystkie). × zamyka render, klatki na dysku zostają.

Rendery dzielą jeden budżet procesów. Obok siebie działa ich najwyżej tyle, ile mówi Settings *Max renders at once* (domyślnie 3); procesy dzielą się między nie (8 → 4 + 4 → 3 + 3 + 2).

Karta po prawej: ALL RENDERS DONE IN

Kiedy powinny być gotowe wszystkie rendery, które działają lub czekają, przy szybkości, z jaką PC renderuje teraz. Pod tym: o której skończą, jak długo już działają i ile klatek zostało. Gdy wszystkie się skończą, karta pokazuje **ALL RENDERS TOOK**: ile to wszystko trwało.

Wiersz sterowania

Element	Co robi
Render	Uruchamia wybrany render (lub kilka wybranych). Klatki, które już są na dysku w porządku, pomija. Po Stop render kontynuuje tam, gdzie skończył (Resume). Gdy render ma klatki na dysku (gotowy, zatrzymany albo z klatkami z wcześniej), pyta: **Only missing** renderuje tylko klatki brakujące lub uszkodzone (pozostałe są na mapie ciemnozielone), **All frames again** renderuje wszystko od nowa, **Cancel** nic.
Pause	Zamraża wszystkie procesy Nuke renderu w połowie klatki (procesor się zwalnia, pamięć zostaje zajęta). Kolejne kliknięcie kontynuuje.
Stop	Zatrzymuje render, kończy jego procesy i usuwa w połowie zapisane klatki i pliki tymczasowe Nuke. Gotowe klatki zostają.
Slate	Czy renderuje się klatka slate i na której klatce (Settings › Job).
Framerange	Klatki tego renderu: zakres z Nuke, chyba że zmieniłeś go w Settings › Job.
Processes · Threads · Cache · Chunk	Z czym render działa. Zmienia się w Settings › Processes and Chunks; liczbę procesów także w trakcie, prawym przyciskiem w System.
Kolorowy pasek	Wynik renderu, patrz niżej.

Kolorowy pasek z wynikiem

Kolor	Co oznacza	Przykład
Zielony	Gotowe, wszystkie klatki są na dysku i przeszły kontrolę.	Done in 2:06. All 97 frames are on disk and pass the check.
Żółty	Gotowe, ale coś nie jest sprawdzone lub wygląda podejrzanie (klatki niesprawdzone, podejrzanie małe; slate nie jest zgłaszany jako mały).	frames not checked · 2 suspiciously small
Czerwony	Błąd: render się nie udał, uszkodzone klatki albo Check pixels znalazł problem (NaN, czarna klatka…).	2 broken frames · Output: Problems shows them
Szary	W toku: *Checking pixels…*, czekanie na wolne procesy i podobne.	Waiting: 1 render already uses the PC

5 Kafelki i mapa klatek



Kafelek	Co pokazuje
Frames	Gotowe / wszystkie klatki renderu, procenty i ile renderuje się teraz.
Elapsed	Jak długo render trwa (bez pauz).
Remaining	Szacowany pozostały czas przy obecnej szybkości.
Speed	Klatki na sekundę i na minutę, wszystkie procesy razem. Najlepsza liczba do porównania ustawień.
Frame time	Ile trwa jedna klatka w jednym procesie i ile klatek liczy się naraz (× N at once).
Nuke processes	Ile procesów renderuje / działa, plus te, które się ładują lub czekają.
Check frames	Kontrola plików: brakujące, puste, ucięte, nieczytelne i podejrzanie małe klatki.
Check pixels	Kontrola pikseli (EXR, PNG, JPEG, TIFF, DPX): czarne, puste i jednokolorowe klatki, klatki takie same jak inna oraz klatka, której rozmiar, kanały lub głębia bitowa różnią się od pozostałych. W EXR także NaN, inf i kompresja.

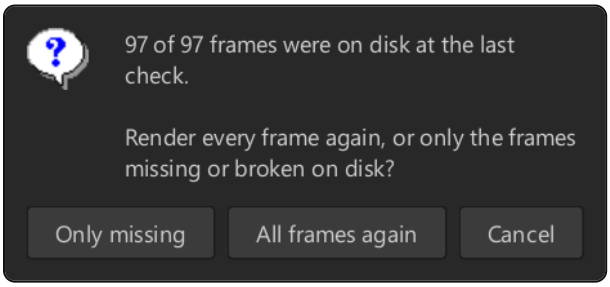
Mapa klatek



Każdy prostokącik to jedna klatka, pogrupowane według chunków (C01, C02…). **Kliknięcie klatki** otwiera ją w podglądzie. Kolory:

czeka (queued) renderuje się gotowe była już na dysku, pominięta (Only missing) podejrzanie mała nieudana / uszkodzona

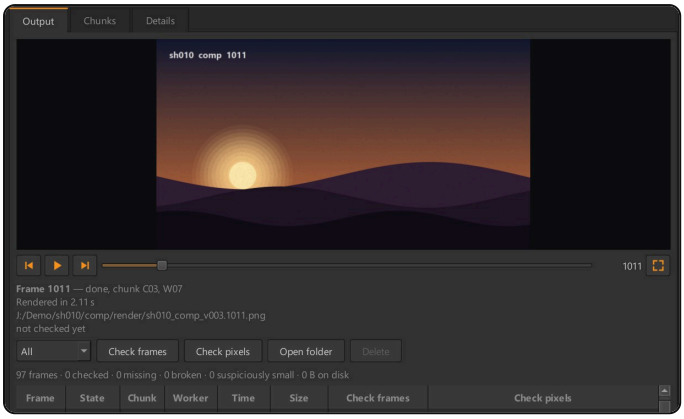
Render again: wszystko albo tylko brakujące



Gdy render ma klatki na dysku i klikniesz **Render again**, pyta: **Only missing** (domyślnie) renderuje tylko klatki brakujące lub uszkodzone; pozostałe pomija i na mapie są **ciemnozielone**. **All frames again** renderuje wszystko od nowa, **Cancel** nic. Film (MOV, MP4) nie pyta: film to jeden plik i renderuje się w całości.

Przy kilku kartach wybranych przez **Shift** pyta raz za wszystkie (np. *27 of 27 frames of 3 renders*…). Klatki, które były na dysku już przy otwarciu albo z przebiegu przed Stopem, są ciemnozielone. W Output filtr **Rendered** pokazuje wtedy tylko klatki z tego przebiegu, **All** wszystkie.

6 Panel Output: podgląd i klatki



Podgląd

- **Kółko myszy**: zoom, **przeciąganie** przesuwa obraz.
- **Dwuklik**: obraz wraca do całej klatki. Gdy już jest cała, podgląd powiększa się na wszystkie trzy panele; z powrotem klawiszem `Esc`.
- Przy zoomie wczytuje się pełna rozdzielczość (także EXR i 4K).
- `Space` uruchamia i zatrzymuje odtwarzanie. Nigdy nie naciska przycisku, więc nie zatrzyma renderu.

Pasek pod podglądem

Poprzednia klatka · **Play/Pause** (odtwarza gotowe klatki w pętli w fps skryptu) · następna klatka · suwak przez wszystkie klatki · numer klatki · **powiększ** (podgląd na trzy panele, `Esc` z powrotem). Pod paskiem: ścieżka, rozmiar, rozdzielczość, kanały i wyniki kontroli klatki.

Przyciski

Element	Co robi
Filtr	**All**: wszystkie klatki naraz: wyrenderowane, pominięte, uszkodzone, *not on disk* i te, które dopiero będą renderowane, oraz klatki na dysku poza Framerange. Pozostałe filtry są jego częściami: **Rendered**: tylko klatki wyrenderowane w tym przebiegu. **Problems**: błędy, podejrzanie małe klatki i znaleziska kontroli pikseli. **Framerange**: tylko klatki wybranego zakresu.
Check frames	Ponownie sprawdza pliki wszystkich klatek: istnieje, nie jest pusty, jest cały i czytelny i nie jest wyraźnie mniejszy od pozostałych. Klatka, której jeszcze nie ma na dysku, zostaje *not on disk* i nie liczy się jako uszkodzona.
Check pixels	W tle sprawdza piksele każdej klatki (EXR, PNG, JPEG, TIFF, DPX): czarne, puste i jednokolorowe klatki, klatki takie same jak inna gdziekolwiek w sekwencji, inny rozmiar, kanały lub głębia bitowa; w EXR także NaN, inf i kompresja. Około 0,1 s na klatkę 4K. Tylko zgłasza, niczego nie renderuje ponownie.
Open folder	Otwiera folder z wyjściem.
Delete	Usuwa wyrenderowane klatki z dysku: wszystkie (domyślnie) lub podany zakres. Najpierw pyta.

Lista klatek

Kolumny: **Frame**, **State**, **Chunk**, **Worker** (który proces ją liczył), **Time**, **Size**, **Check frames**, **Check pixels**. Z filtrem All wszystkie klatki są na liście od początku, a ich stan zmienia się w trakcie renderu. Po **Render again** klatki nie znikają: zostają jako *on disk, queued again* ze starym rozmiarem i czasem (można je oglądać i odtwarzać), dopóki nowy render ich nie nadpisze. Klatki na dysku poza wybranym Framerange zostają na liście z dopiskiem *not in Framerange*.

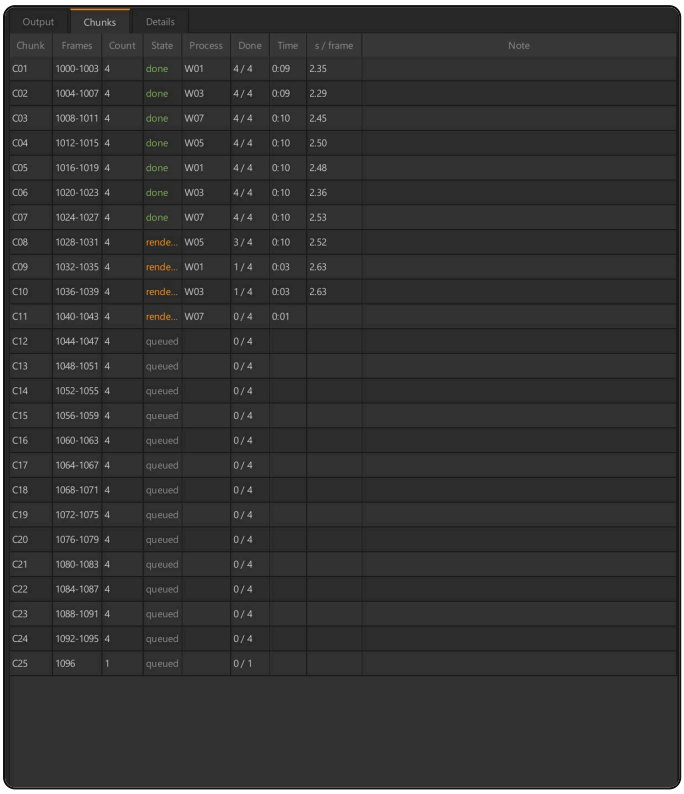


Prawy przycisk na klatce (lub na kilku wybranych):
Render this frame / batch: renderuje ponownie tylko te klatki (gdy render działa, dodają się do niego).
Delete this frame / batch: usuwa je z dysku (tylko gdy render nie działa).

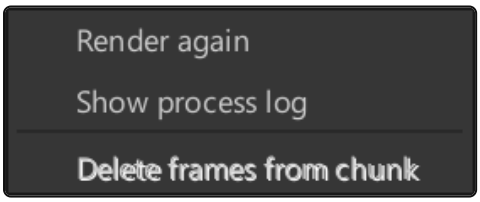
> **MOV / MP4 / MXF:** zamiast listy klatek pojawia się jeden plik filmu ze szczegółami (kodek, profil, fps, rozdzielczość, timecode, liczba klatek wobec ustawień Write'a) i odtwarzacz z lupą. Przyciski Check video, Open folder, Delete video. Filmu nie da się podzielić, renderuje go więc jeden proces ze wszystkimi wątkami; tak pokazują to też Settings (Processes 1, zablokowane). Ustawienia pozostałych renderów się nie zmieniają.

7 Panele Chunks i Details

Chunks



Jeden wiersz na chunk: **Chunk** (C01…), **Frames** (zakres), **Count**, **State**, **Process** (który go liczy), **Done**, **Time**, **s / frame** i **Note** (dlaczego chunk był powtórzony i podobne).



Prawy przycisk: Render again (chunk ponownie), **Show process log** (log procesu, który go liczył), **Delete frames from chunk** (tylko gdy render nie działa).

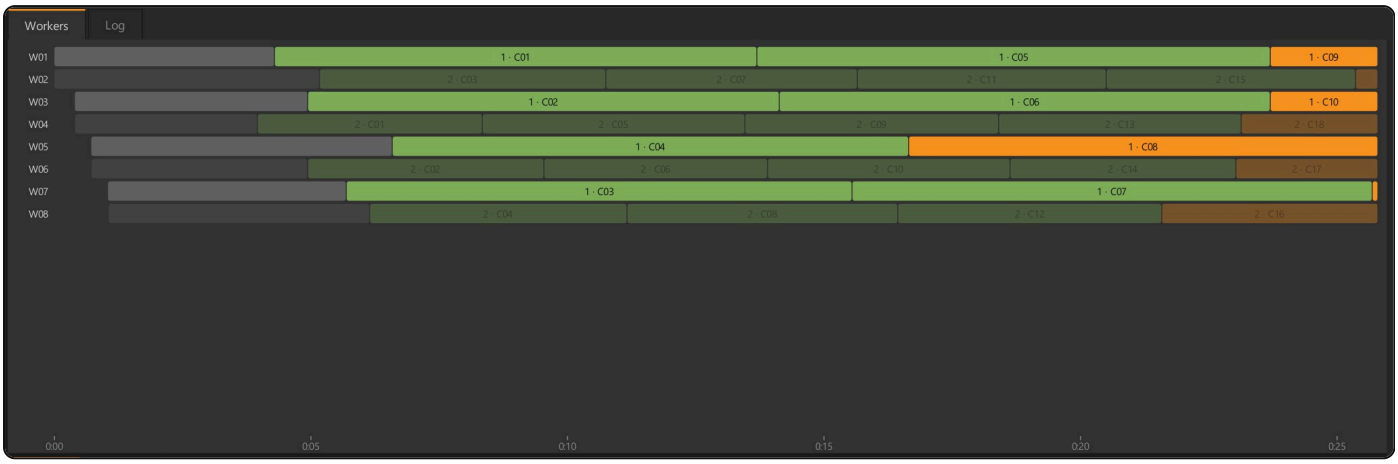
Details



Wszystko o renderze w jednym miejscu: skrypt i jego snapshot, Write'y i ścieżki wyjścia, format, fps, zakres, wersja Nuke, z jakimi Processes × Threads × Cache działa, czasy i ścieżki do logów. Przydaje się, gdy chcesz wiedzieć, co dokładnie się renderowało.

8 Panele Workers i Log

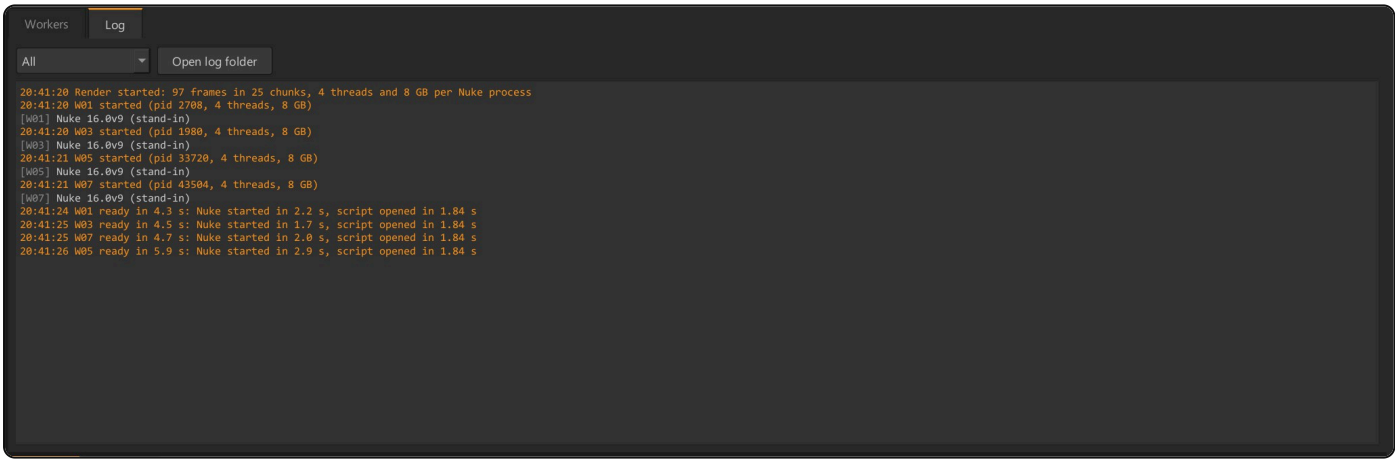
Workers: oś czasu procesów



Jeden wiersz na proces Nuke (W01–W08), czas od lewej do prawej. ⬛ **szary**: Nuke startuje i otwiera skrypt (także przy wymianie na świeży Nuke, z etykietą *again: fresh Nuke*). 🟩 **zielony**: gotowe chunki (z numerem chunka). 🟧 **pomarańczowy**: chunk, który właśnie się renderuje. Chunki innych renderów są przyciemnione (Settings › *Dim other renders in Workers*). Od razu widać, czy któryś proces stoi, startuje ponownie albo długo otwiera skrypt.

Prawy przycisk na wierszu procesu otwiera to samo menu co w panelu System (rozdział 9).

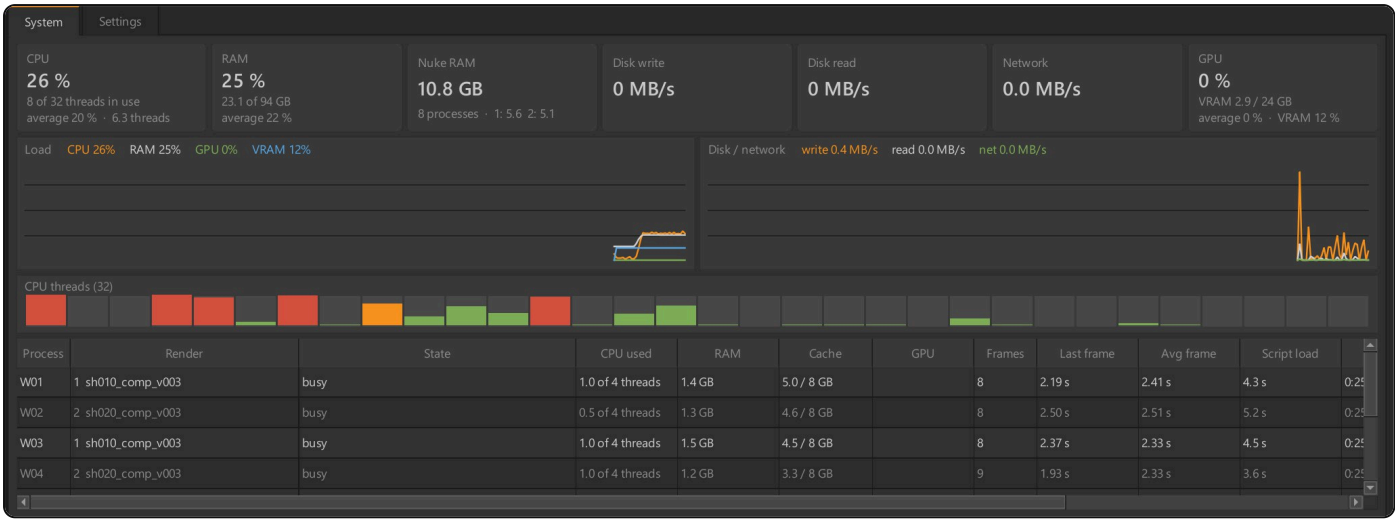
Log



Komunikaty JKrender i wyjście każdego procesu Nuke z czasem. Filtr: **All**, **JKrender only** (tylko komunikaty JKrender), **Errors and warnings**. Open log folder otwiera folder z logami tego renderu: snapshot skryptu, `job.json` i `worker_NN.log` każdego procesu.

> Każdy proces podaje w logu, po jakim czasie był gotowy, z podziałem na **start Nuke** i **otwarcie skryptu**. Gdy start trwa 15 s lub dłużej, JKrender ostrzega. Wolny start to zwykle licencja albo skrypty startowe pipeline'u, nie sam skrypt.

9 Panel System



Kafelki i wykresy

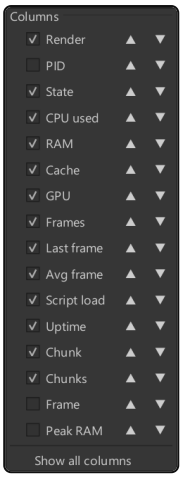
CPU (użycie i wątki), **RAM** (całkowita), **Nuke RAM** (ile biorą procesy Nuke), **Disk write**, **Disk read**, **Network**, **GPU** (użycie i VRAM). Wykres **Load** pokazuje CPU, RAM, GPU i VRAM w %, wykres **Disk / network** transfery w MB/s. **CPU threads**: jeden słupek na wątek procesora, więc widać, czy cały PC jest obciążony.

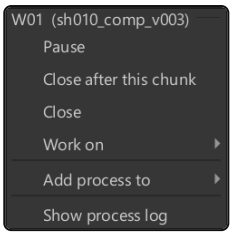
Tabela procesów

Kolumna	Znaczenie
Process	Numer procesu (W01…).
Render	Nad którym renderem pracuje.
State	Co robi: loading, busy, idle, paused, closing… *busy → fresh Nuke (RAM 91 %)*: po swoim chunku dostanie świeży Nuke, bo RAM jest przy limicie (rozdział 13). Na co czeka, np. *wait: VRAM 75 → 14 % (3 starting)*: teraz 75 %, kolejny proces zmieści się poniżej 14 %, bo 3 procesy dopiero startują i JKrender trzyma dla nich miejsce; tooltip mówi więcej.
CPU used	Ile wątków procesora proces naprawdę wykorzystuje.
RAM · Cache	Rzeczywista pamięć procesu i jego cache Nuke wobec limitu.
GPU	Użycie karty graficznej i VRAM tego procesu.
Frames · Last frame · Avg frame	Ile klatek zrobił, ile trwała ostatnia i średnio.
Script load	Po jakim czasie Nuke wystartował i otworzył skrypt.
Uptime · Chunk · Chunks	Jak długo działa, na którym chunku jest i ile ich zrobił.
PID · Frame · Peak RAM	Ukryte kolumny: ID procesu w Windows, bieżąca klatka, najwyższa RAM.

Kliknięcie nagłówka tabeli otwiera listę kolumn: zaznaczeniem kolumnę pokażesz lub ukryjesz, strzałkami zmienisz kolejność. Kolumny można też przeciągać za nagłówek. Szerokości i kolejność zapisują się przy zamknięciu okna.

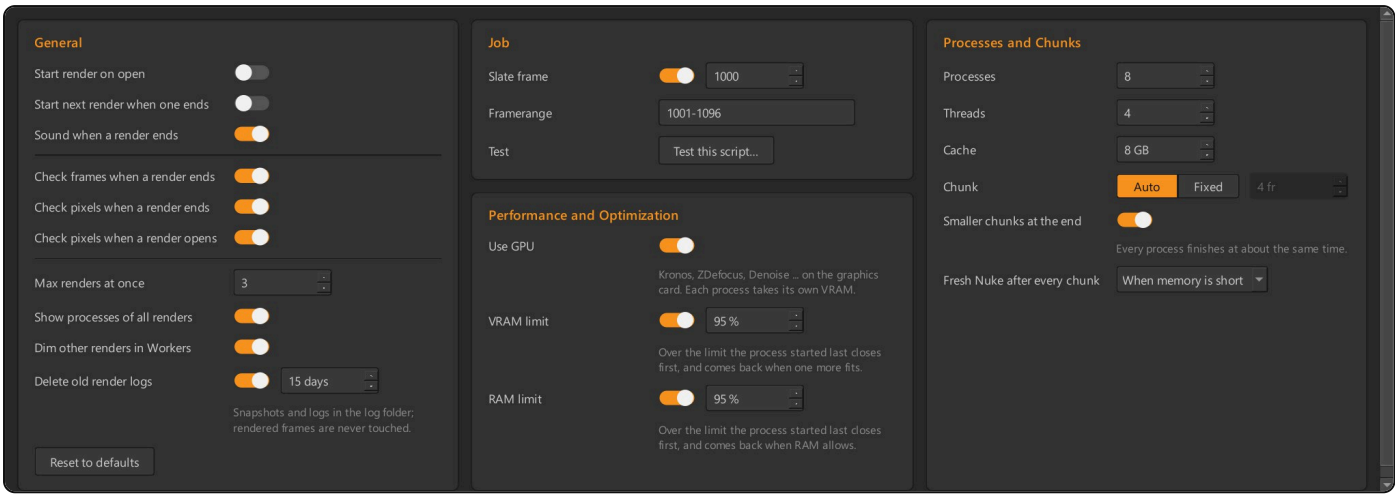
Prawy przycisk na procesie (w tabeli lub na wierszu w Workers) otwiera menu po prawej. Przy kilku działających renderach *Add process to* ma podmenu z wyborem renderu.





Pozycja menu	Co robi
Pause / Resume	Zatrzymuje lub puszcza tylko ten proces.
Close after this chunk	Proces dokończy swój chunk i się zamknie. **Processes** zmniejsza się o 1 i tak zostaje.
Close	Zamyka proces od razu. Jego chunk wraca do kolejki. Na jego miejsce może wejść inny.
Work on › render	Przenosi proces na inny działający render.
Add process	Uruchamia jeszcze jeden proces (także kliknięciem w puste miejsce tabeli). **Processes** zwiększa się o 1 i tak zostaje. Limity RAM i VRAM nadal obowiązują.
Start again	Dla zamkniętego procesu: uruchamia go ponownie.
Show process log	Otwiera log tego procesu.

10 Settings: każde ustawienie



Ustawienia zapisują się od razu. Każde ma podpowiedź po najechaniu myszą. **Reset to defaults** przywraca wszystko do wartości domyślnych, także kolumny tabel; to, czego JKrender nauczył się o skryptach, zostaje.

General

Ustawienie	Co robi	Domyślnie
Start render on open	Render startuje od razu po otwarciu okna, bez klikania Render.	wył.
Start next render when one ends	Gdy render się skończy lub nie uda, sam startuje następny jeszcze nieuruchomiony render w kolejności kart. Render, który zatrzymałeś, niczego nie uruchamia.	wył.
Sound when a render ends	Dźwięk po skończeniu renderu, gdy dobiegną włączone kontrole. Inny dźwięk przy błędzie, uszkodzonych klatkach lub problemie w pikselach. Zatrzymany render jest cicho.	wł.
Check frames when a render ends	Po skończeniu renderu (także po Stop) raz każda wyrenderowana klatka: jest, nie jest pusta, jest cała i czytelna, nie jest wyraźnie mniejsza od pozostałych. W trakcie renderu nic nie czyta folderu wyjścia. Wyłączone: uszkodzona klatka liczy się jako gotowa, dopóki nie naciśniesz Check frames.	wł.
Check pixels when a render ends	Zaraz po kontroli klatek w tle sprawdza piksele wszystkich klatek (czarne, jednokolorowe, takie same, inny format; w EXR także NaN i inf). Tylko zgłasza.	wł.
Check pixels when a render opens	Ta sama kontrola klatek, które już są na dysku, zaraz po otwarciu renderu. Render ją przerywa.	wł.
Max renders at once	**1**: rendery idą po kolei, następny zaczyna się ładować, gdy poprzedni kończy ostatnie chunki (najpierw gotowy pierwszy shot). **2–3**: działają obok siebie i dzielą się procesami (łącznie nie szybciej, ale z każdego shota widzisz klatki wcześnie).	3
Show processes of all renders	Workers i System pokazują procesy wszystkich renderów. Wyłączone: tylko wybranego.	wł.
Dim other renders in Workers	Na osi czasu świecą chunki wybranego renderu, pozostałe są przyciemnione.	wł.
Delete old render logs	Przy otwarciu JKrender usuwa foldery logów starsze niż ustawiona liczba dni. Klatek nigdy.	wł., 15 dni

Grupy w General oddzielają cienkie linie. Przy ponownym renderze oś czasu Workers pokazuje zawsze tylko nowy przebieg.

Job (dotyczy renderu wybranego u góry)

Ustawienie	Co robi	Domyślnie

Slate frame	Projekt zaczyna się za slatem (np. 1001, slate na 1000). Włączone: każdy render dodatkowo renderuje slate na tej klatce.	wł., 1000
Framerange	Klatki renderu, np. `1001-1100` lub `1001-1010,1050`. Każdy render ma swój. Puste pole przywraca zakres z Nuke. Zatwierdzasz Enterem lub kliknięciem gdzie indziej. W trakcie renderu jest zablokowane. Klatki na dysku zostają.	z Nuke
Test › Test this script...	Mierzy ustawienia dla tego PC na klatkach z całego shota i pokazuje najszybsze (rozdział 12).	—

Performance and Optimization

Ustawienie	Co robi	Domyślnie
Use GPU	Uruchamia Nuke z `--gpu`, więc nody GPU (Kronos, ZDefocus, Denoise...) liczy karta graficzna. Lekki skrypt 4K z Kronosem był 3× szybszy. Każdy proces ma własną VRAM.	wł.
VRAM limit	Powyżej limitu działa mniej procesów. Przy 97 % od razu zamyka się proces uruchomiony jako ostatni; tak samo, gdy karta jest pełna i żadna klatka długo się nie kończy. W granicach 5 % od limitu pomaga Fresh Nuke after every chunk. Rozdział 13.	wł., 95 %
RAM limit	Powyżej limitu działa mniej procesów: najpierw zamyka się uruchomiony jako ostatni. Gdy pamięć się zwolni, wraca. Windows nie zacznie więc swapować.	wł., 95 %

Processes and Chunks

Ustawienie	Co robi	Domyślnie
Processes	Procesy Nuke dla wszystkich renderów razem.	8
Threads	Wątki na proces (`-m`).	4
Cache	Limit cache Nuke na proces w GB (`-c`). Ile proces naprawdę bierze, pokazuje System › RAM. Za mały cache spowalnia render: Nuke wtedy wyrzuca i liczy ponownie.	wg RAM
Chunk: Auto / Fixed	**Auto**: około 4 chunki na proces, najwyżej 10 klatek (46 klatek przy 8 procesach → 2, 96 → 3, 320+ → 10). **Fixed**: rozmiar wpisujesz po prawej. Dotyczy chunków, które jeszcze się nie zaczęły.	Auto
Smaller chunks at the end	Pod koniec renderu chunki zmniejszają się aż do pojedynczych klatek, więc wszystkie procesy kończą mniej więcej razem.	wł.
Fresh Nuke after every chunk	Po swoim chunku proces zamyka Nuke i otwiera skrypt w nowym, z RAM i VRAM znów na początku. Kosztuje to jedno wczytanie skryptu. **When memory is short**: tylko gdy RAM lub VRAM jest 30 s bez przerwy w granicach 5 % od limitu, i to po jednym procesie. **Always**: po każdym chunku. **Off**: nigdy. Rozdział 13.	When memory is short

> Processes, Threads i Cache są w trakcie renderu zablokowane. Dla filmu pokazują 1 proces ze wszystkimi wątkami. Liczbę procesów w trakcie zmienisz prawym przyciskiem w System (*Add process*, *Close after this chunk*).

11 Jak ustawić render

Zalecany sposób przy nowym lub ciężkim skrypcie:

1	2	3	4	5
Wyślij Write Ctrl+Alt+R	**Create tests** ustawienia wg PC	**Start testu** Round 1 i Round 2	**Use these settings** przejmuje zielony wiersz	**Render** i obserwuj System

Kroki 2–4 są w oknie Test this script. Lekkie skrypty śmiało od razu na domyślnych 8 × 4; test opłaca się, gdy render potrwa dziesiątki minut.

Orientacyjne ustawienia (Ryzen 9950X, 32 wątki, 94 GB RAM, RTX 3090 24 GB)

Typ skryptu	Processes × Threads	Dlaczego	Zmierzone
Lekki comp, keying, grade	**8 × 4**	Jeden Nuke nie wykorzysta wszystkich wątków, więcej procesów tak.	testowy skrypt 4K: 8 × 4 najszybsze
Ciężki comp (Defocus, Denoise, wiele warstw)	**4 × 8** lub **2 × 16**	Jeden proces wykorzysta więcej wątków. 8 procesów wyczerpie RAM, procesy się zamykają.	ciężki comp: 4 × 8 z cache 12 GB 17 fr/min, 2 × 16 16 fr/min
Skrypt, któremu rośnie VRAM (nody GPU)	**1–2 × 32/16**	VRAM rośnie z każdą klatką; więcej procesów nie zmieści się na karcie.	VRAM +1,7 GB na klatkę → 1 × 32

Jak poznać, że ustawienie nie jest dobre

Mało pamięci

System › RAM jest przy limicie, w State jest *wait: RAM …*, procesy się zamykają i startują ponownie (szare bloki w Workers). **Rozwiązanie:** mniej procesów z większą liczbą wątków lub mniejszy Cache.

Pełna karta graficzna

VRAM przy 95 %, procesy zamykają się z powodu VRAM. JKrender sam zaproponuje mniej procesów z większą liczbą wątków (ale sam niczego nie zmienia). **Rozwiązanie:** mniej procesów z większą liczbą wątków (Test this script pokaże ile) albo wyłączyć Use GPU.

Niewykorzystany CPU

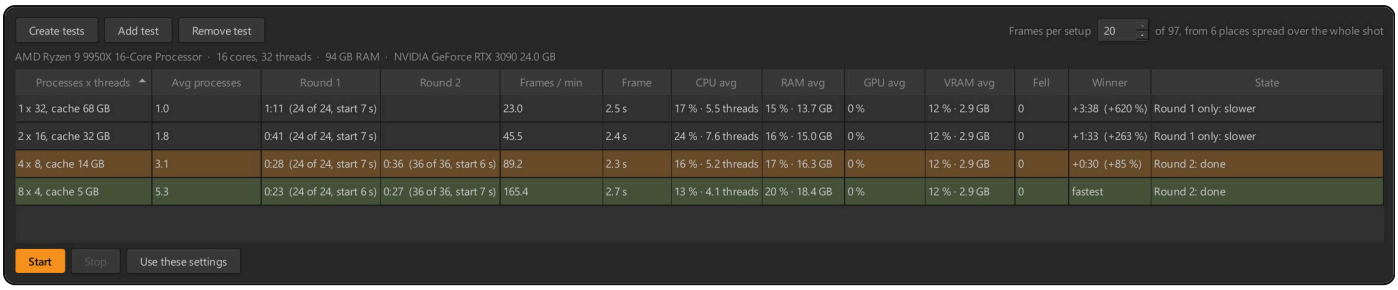
CPU threads są w dużej części puste, a RAM ma zapas. **Rozwiązanie:** dodaj proces (prawy przycisk › Add process) albo zwiększ Threads.

Długie otwieranie skryptu

Szare bloki w Workers są długie, Script load jest wysoki. Przy starcie Nuke'i walczą o licencje i skrypty startowe. **Rozwiązanie:** przy dłuższych chunkach mniej to widać; ze samym skryptem zwykle nie ma to związku.

> **Najlepszą miarą jest kafelek Speed** (klatki na minutę). Porównuj go między ustawieniami na tym samym skrypcie albo zostaw to *Test this script*.

12 Test this script



Gotowy test: zielony wiersz jest najszybszy, pomarańczowy drugi. Winner pokazuje, o ile dłużej trwałby cały shot z pozostałymi.

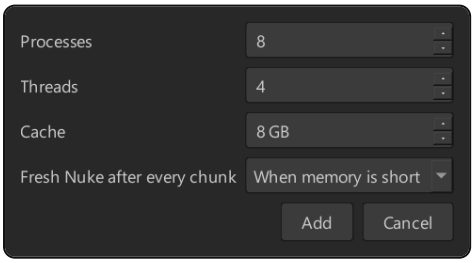
Przyciski u góry

Create tests sprawdza komputer (wiersz pod przyciskami: CPU, rdzenie i wątki, u Intela rdzenie wydajne i energooszczędne, RAM, grafika) i tworzy **7 ustawień** od 1 procesu ze wszystkimi wątkami po tyle procesów, ile PC udźwignie: każdy proces co najmniej 4 wątki i co najmniej 4 GB cache. Na 32 wątkach i 94 GB: 1×32, 2×16, 3×10, 4×8, 5×6, 6×5, 8×4. Powyżej 64 wątków proces dostaje najwyżej 64 (jedna grupa procesorów w Windows). Zastępuje wszystko w tabeli.

Add test dodaje własne ustawienie (Processes, Threads, Cache i Fresh Nuke after every chunk), Remove test usuwa wybrane. **Prawy przycisk** na wierszu: **Edit test**. Własne testy są zapamiętywane. Tabela pokazuje Fresh Nuke w nazwie ustawienia: *4 x 8, cache 12 GB, fresh every chunk* albo *never fresh*; bez dopisku to When memory is short.

Cache każdego ustawienia to cała RAM dla Nuke (limit RAM minus rezerwa dla Windows) podzielona przez liczbę procesów, minus 4 GB, które proces bierze poza cache. Nie ma znaczenia, co akurat działa.

Frames per setup (u góry po prawej): ile klatek każde ustawienie renderuje w Round 2. Gdy shot ma mniej klatek, pokaże to i renderuje wszystkie.



Jak przebiega test

Klatki brane są z **6 miejsc rozłożonych po całym shocie** (od pierwszej do ostatniej klatki), w każdym kilka klatek po kolei. **Wszystkie ustawienia w rundzie renderują te same klatki.**

Round 1: każde ustawienie krótko, 3 klatki na proces ustawienia z największą liczbą procesów (na 32 wątkach 24 klatki), najwyżej 3 minuty. **Round 2:** dwa najszybsze (i każde w granicach 5 % od najlepszego) ponownie z Frames per setup, co najmniej 4 klatki na proces.

Frames / min liczy się, gdy procesy już działają: start Nuke i pierwsza klatka każdego procesu się nie liczą; w długim renderze zdarzają się tylko raz.

Kolumny: **Avg processes** (ile procesów średnio renderowało), **Round 1**, **Round 2** (czas rundy, klatki, start), **Frame**, **CPU / RAM / GPU / VRAM avg**, **Fell** (procesy, które padły), **Winner**, **State**. Kliknięcie nagłówka sortuje, kolumny można rozciągać i przesuwać.

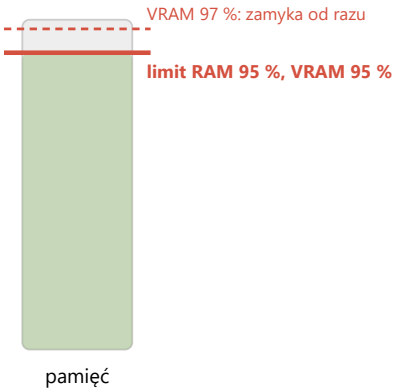
Use these settings przenosi zielony wiersz do Settings, także Fresh Nuke. Klatki idą do folderu testowego i są usuwane; render hooki Write'a w teście się nie uruchamiają. Tabele zostają w `~/.nuke/JKrender/settings/test_results`.

Okno **Add test** (i takie samo **Edit test** z prawego przycisku).

> Podczas testu nic innego nie powinno się renderować: test mierzyłby dwa rendery walczące o PC. Obok renderu test się nie uruchomi.

13 Pilnowanie RAM i VRAM

JKrender pilnuje pamięci, żeby Windows nie zaczęły swapować, a karta graficzna się nie zapełniła.



30 s bez przerwy w granicach 5 % od limitu
Po swoich chunkach procesy kolejno, po jednym, dostają świeży Nuke (Fresh Nuke).

Powyżej limitu
Zamyka się proces uruchomiony jako ostatni: od razu, gdy otwiera skrypt, inaczej po chunku.

Powrót
Wraca naraz tyle procesów, ile się zmieści; startują po kolei.

Powtórne przeciążenie
Czeka 5 s, potem 30, 60 i 150 s, gdy powtórzy się w ciągu 10 minut.

Żeby proces wrócił, pod limitem musi zmieścić się jeden proces takiej wielkości, jaką miały procesy w tym renderze (do tego czasu: jego Cache + 4 GB), z rezerwą 2 % i z tym, co działające procesy jeszcze zabiorą. Na co się czeka, pokazuje kolumna **State** w System, np. *wait: VRAM 75 → 69 %*: VRAM jest na 75 % i musi spaść poniżej 69 %, żeby zmieścił się kolejny proces. *wait: VRAM 75 → 14 % (3 starting)*: trzy procesy dopiero startują, a JKrender trzyma dla nich miejsce w pełnej wielkości. Inne stany: *wait: 25 s* (proces właśnie padł), *wait: W03 1st frame* (czeka na pierwszą klatkę W03, żeby było widać, ile proces bierze), *wait: VRAM ok, checking* (miejsce jest, sprawdza, czy się utrzyma).

Pełna karta i nic nie idzie: gdy VRAM jest powyżej limitu, a żaden proces nie kończy klatki dłużej niż 4× zwykły czas klatki (co najmniej minutę), Nuke najpewniej przeszedł na CPU i wszystkie procesy czekają. Wtedy od razu zamyka się proces uruchomiony jako ostatni, a jego chunk wraca do kolejki.

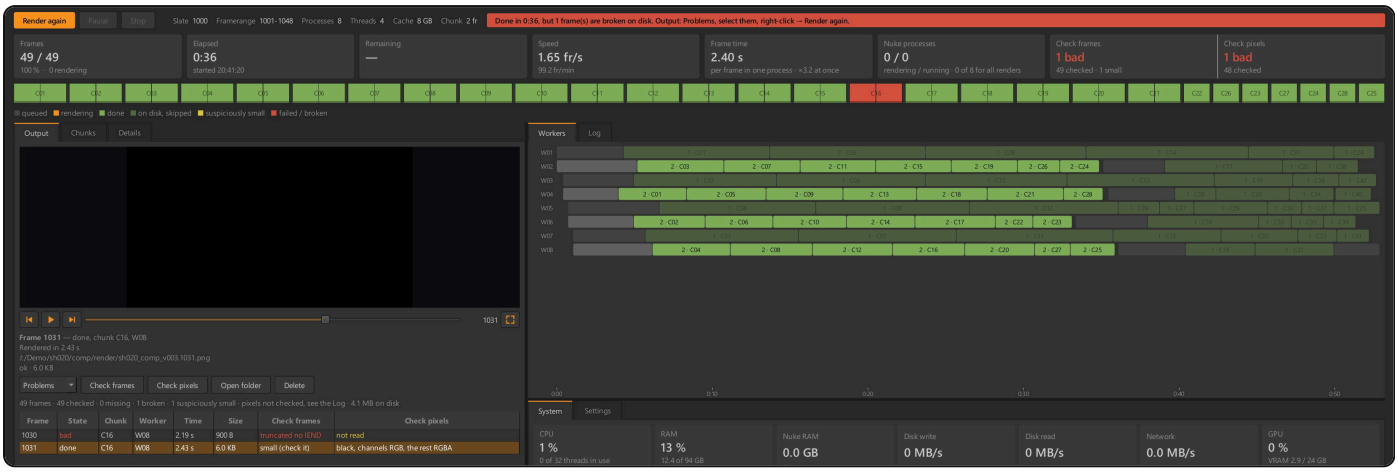
Fresh Nuke after every chunk

Podczas renderu Nuke trzyma pamięć, której już nie potrzebuje: proces ciężkiego skryptu może w pierwszych ~20 klatkach urosnąć z 2 do 7 GB VRAM. Świeży Nuke zaczyna znów mały, więc na karcie zmieści się więcej procesów. W domyślnym **When memory is short** dzieje się to tylko wtedy, gdy RAM lub VRAM jest **30 s bez przerwy** w granicach 5 % od limitu (pojedynczy skok nie wystarczy), i to **po jednym procesie**: następny dopiero wtedy, gdy nowy Nuke wyrenderuje pierwszą klatkę, a pamięć wciąż jest przy limicie. Pozostałe w tym czasie renderują dalej. Proces, który dostaje świeży Nuke, pokazuje w State *busy → fresh Nuke (VRAM 93 %)*; przy ostatnich chunkach nic się nie wymienia.

Limitu VRAM na proces nie ma. Cache (`-c`) ogranicza tylko RAM cache Nuke. Pamięci karty graficznej jednego procesu ograniczyć się nie da: Windows tego nie potrafi, a ustawienia cache Blink w Nuke w renderze nie działają (zmierzone). Dlatego JKrender steruje VRAM liczbą procesów i świeżymi Nuke'ami. Cache to limit cache, nie cała pamięć procesu; prawdziwa liczba jest w System › RAM.

14 Kontrole, dźwięki, awarie i dane

Kontrola wyjścia



Render z uciętą klatką (1030) i czarną klatką (1031), obie znalezione ręcznym Check frames i Check pixels: czerwony pasek, czerwone klatki na mapie, filtr Problems pokazuje tylko je.

Kontrola	Kiedy	Czego szuka
Check frames	Po skończeniu renderu i po Stop (można wyłączyć) oraz przyciskiem. Zawsze przy otwarciu renderu i przed Renderem.	Brakujące, puste, ucięte i nieczytelne pliki (EXR, PNG, JPEG, DPX), podejrzanie małe klatki.
Check pixels	Po skończeniu renderu, przy otwarciu (można wyłączyć) i przyciskiem.	EXR, PNG, JPEG, TIFF i DPX: czarne, puste i jednokolorowe klatki, klatki takie same jak inna, inny rozmiar, kanały i głębia bitowa. W EXR także NaN, inf i kompresja. Inne formaty: tylko identyczne pliki.
Movie check	Dla MOV, MP4 i MXF.	Kodek, profil, fps, rozdzielczość, timecode i liczba klatek wobec ustawień Write'a.

Dźwięk po renderze

Po renderze kontrole idą w kolejności **Check frames** → **Check pixels**; dźwięk rozlega się, gdy dobiegną te włączone (przy obu wyłączonych od razu po renderze).

Wesoły dźwięk: wszystko w porządku. **Dźwięk błędu**: render się nie udał, ma uszkodzone klatki albo kontrola pikseli znalazła problem. Ręcznie zatrzymany render jest cicho.

Gdy JKrender padnie

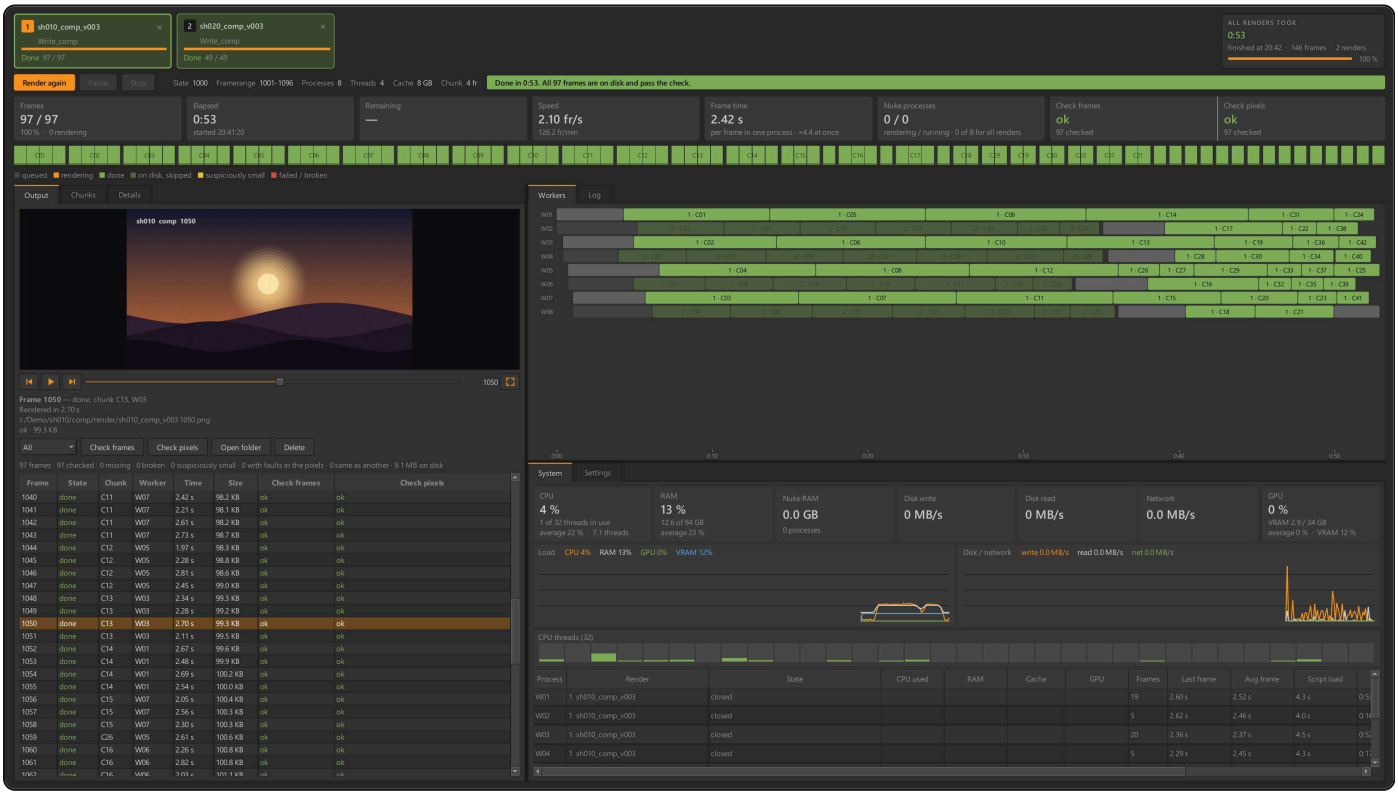
Karty wrócą przy następnym otwarciu JKrender (np. kolejnym renderem wysłanym z Nuke). To, co jest na dysku, się liczy, a pozostałe procesy Nuke są kończone. Render, który działał, czeka na Render, a potem dorenderuje, czego brakuje. Gdy zamkniesz okno normalnie, nic nie wraca.

Dane renderu

Każdy render zapisuje do swojego folderu logu `report.json`: CPU, RAM i kartę graficzną tego PC, ustawienia, szybkość, szczyty pamięci, zamykanie z powodu limitów, start Nuke, otwarcie skryptu i rodzaje nodów w skrypcie. **Nie ma w nim nazw skryptów, ścieżek ani błędów**; skrypt rozpoznaje się tylko po ID, które jest takie samo we wszystkich jego wersjach. Gdy trafisz na błąd, wyślij folder logu renderu.

15 Mysz i klawisze: ściąga

Gdzie i co	Wynik
`Ctrl+Alt+R` w Nuke	Wysyła wybrane Write'y do JKrender.
Kliknięcie karty renderu	Wybiera render; całe okno pokazuje jego dane.
`Shift` + kliknięcie kart	Wybiera kilka renderów dla Render, Render again, Pause, Stop.
Kliknięcie klatki na mapie klatek	Otwiera klatkę w podglądzie.
`Space`	Odtwórz / zatrzymaj podgląd (nigdy nie zatrzyma renderu).
Kółko / przeciąganie w podglądzie	Zoom / przesunięcie obrazu.
Dwuklik w podglądzie	Cała klatka; gdy już jest cała, duży podgląd.
`Esc`	Powrót z dużego podglądu.
Prawy przycisk na klatce (Output)	Render this frame / batch, Delete this frame / batch.
Prawy przycisk na chunku (Chunks)	Render again, Show process log, Delete frames from chunk.
Prawy przycisk na procesie (System, Workers)	Pause, Close after this chunk, Close, Work on, Add process, Start again, Show process log.
Prawy przycisk w puste miejsce w System	Add process.
Kliknięcie nagłówka tabeli System	Wybór i kolejność kolumn.
Przeciąganie granicy paneli	Zmienia rozmiar paneli (zapisuje się).
Framerange + `Enter`	Zatwierdza nowy zakres (Settings › Job).



Gotowy render: zielony pasek, karta w prawym górnym rogu pokazuje, jak długo trwały rendery.

16 Gdy coś nie działa

Problem	Co z tym zrobić
Procesy Nuke długo startują (30–50 s)	Zwykle start Nuke (licencja, skrypty startowe pipeline'u) w chwili, gdy PC renderuje. Log pokazuje podział *Nuke started in … / script opened in …*.
Procesy się zamykają i startują ponownie	Zabrakło RAM lub VRAM (State w System, Log). Mniej procesów z większą liczbą wątków albo mniejszy Cache; Test this script pokaże, co jest najszybsze.
W State jest *busy* → *fresh Nuke*	RAM lub VRAM jest od 30 s przy limicie; po chunku proces dostanie świeży Nuke (Settings › Fresh Nuke after every chunk). Gdy dzieje się to ciągle, działa więcej procesów, niż się mieści.
Cały render jest wolniejszy niż ostatnio	Sprawdź Cache: za mały (np. 4 GB przy ciężkim skrypcie) oznacza, że Nuke wyrzuca i liczy ponownie, a klatki trwają kilka razy dłużej.
Skrypt otwiera się w Nuke z błędem knoba	Np. *LensDistortion2.serialiseKnob: Failed to load* (nod z innej wersji, brakujący plugin). JKrender renderuje dalej, tak jak Nuke w GUI; komunikat jest w logu procesu.
Żółty pasek „not checked"	Kontrola klatek jest wyłączona (Check frames when a render ends) albo się nie odbyła. Naciśnij Check frames.
Czerwony pasek	Output › filtr Problems pokaże, które klatki. Prawym przyciskiem *Render this frame / batch*. Pasek przelicza się także po ręcznym Check frames i Check pixels.
Brak podglądu EXR	Brakuje numpy/OpenEXR. Usuń `lib/DECLINED` i zrestartuj Nuke albo zainstaluj je według README.
Render nie zaczął się od razu	Stan Waiting: inne rendery już obciążają PC (Max renders at once) albo czeka się na pamięć.
Czyszczenie cache Nuke nie działa	Po aktualizacji JKrender trzeba raz zrestartować Nuke.
Nie wiem, co się renderowało	Panel Details i Log › Open log folder (snapshot skryptu i job.json).

> **JKrender jest w wersji alpha.** W testach działa, ale licz się z drobiazgami. Gdy trafisz na błąd, wyślij autorowi folder logu renderu (Open Renders Folder) i opis, co się stało.